Specifications - Projet intégrateur Group 6A

BERTHELOT Sacha

February 14, 2025

Contents

1 Game mechanics and interactions **1**
1.1 Basic rules . . . 1
1.2 Interactions between players and server . . . 4

2 Key elements and choices **5**
2.1 Programming . . . 5
2.1.1 Mobile development . . . 5
2.1.2 Web development . . . 5
2.1.3 Interactions . . . 5
2.2 System . . . 6
2.2.1 Interfaces . . . 6
2.2.2 Client / Server communications . . . 6
2.3 Database . . . 6
2.4 Network . . . 6
2.4.1 Implementation and communication protocol tests . . . 6
2.4.2 Implementation of clients and server . . . 6
2.4.3 Communications safety . . . 7
2.4.4 Network's infrastructure . . . 7
2.5 Communication / Marketing . . . 7
2.5.1 Packaging . . . 7
2.5.2 Communication around project . . . 8

3 Tools and software used with justifications **8**
3.1 Development tools . . . 8
3.2 Network tools . . . 9
3.3 Graphic tools . . . 9
3.4 Management tools . . . 10

4 Project manager's role seen by the team **11**

5 Work teams and tasks of the project **12**
5.1 Different teams . . . 12
5.2 Tasks organization and distribution . . . 12

Introduction

The objective of the course "Projet intégrateur" is to develop a board game in a team of eight to eleven people first on PC support then later on a Web or Mobile platform. To achieve this goal, we need to gather all the knowledge we acquired during the last five semesters and put them into one project.

To keep track of the progress of the team and to give tasks to each member, project management tools are used throughout the different development phases. The project manager is in charge of all these documents and also must rearrange their content if needed depending on the unavailability and problems encountered.

In this report, we'll explain each point that will be treated in the project and what tool we would like to be use to achieve each tasks until the final presentation.

1 Game mechanics and interactions

1.1 Basic rules

The idea of our project is to make a traditional board game in the style of a Monopoly. The main objective is to buy properties and create buildings and hotels to increase their rent. When other players land on a property that is the one of someone else, they have to pay them the associated rent depending on the state of the spot they land on. The game ends when only one player still has money and the others went bankrupt.

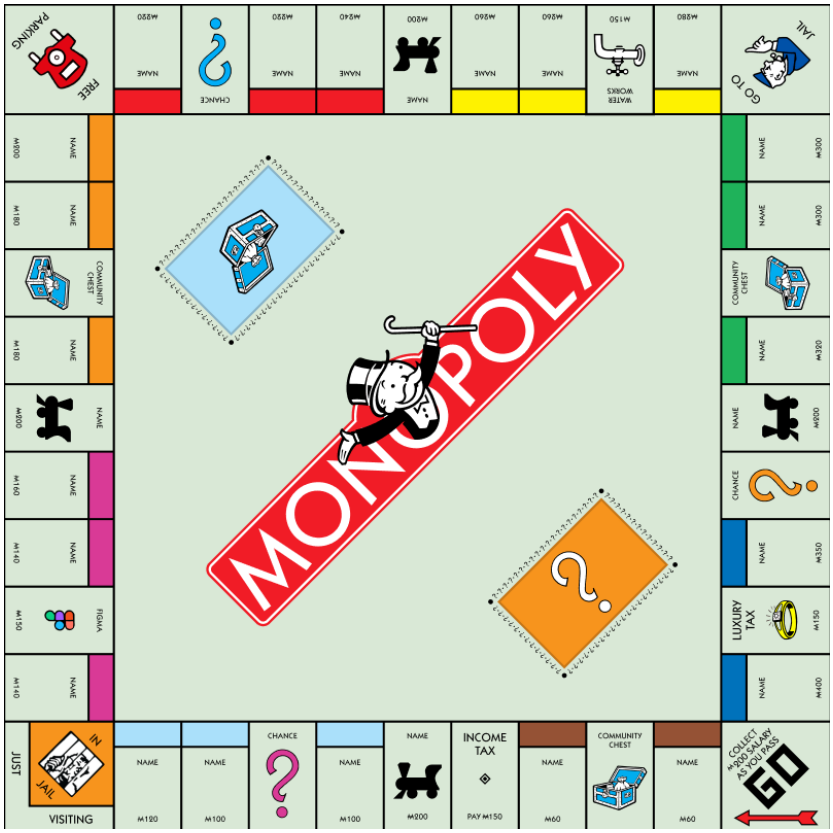

Figure 1: Example of a Monopoly Board

To help the players get money without having properties, they can land on different spaces that give them bonuses or penalties. In the regular game they're called "Chance" and "Community Chest"

and consist of taking the top card of the respective deck and do what they say immediately or later depending on the instructions. Sometimes they have to pay money, sometimes they get money or even special cards. To get a cash bonus, players get money every time they pass by the "Start" space to help them buy new properties.

In addition to the bonuses, players will sometimes land on spaces that will make their journey slightly more difficult:

- "Go to jail: Directly sends the player to jail where they have to stay for two rounds without moving unless they pay a rent to get out earlier or if they got a "Get out of prison" card from the Chance deck. (While in jail, players can still get paid by other players landing on their properties).
- "Income tax": A space at the end of the board where if players land on it must pay an expensive rent to the bank. Penalizing them while being between two of the most expansive properties of the game.

All the properties are divided in different color groups, and if a player possess all the spaces of a same color, they can build houses and hotels for them which will increase their rent and make it harder for other players to pay it. It needs to build four houses before building a hotel, the final upgrade of a property.

To move on the board, players must throw two dices numerated from one to six. If they make a different score on each dice, they sum both of the numbers and travel through the number indicated. If they make the same result on both dices, they also sum the numbers and travel the associated number but can play another turn. If they have three consecutive "double scores", they have to go in jail for getting across too many spaces at once.

Some games can last multiple hours, that is why we want to put a time limit to the game where players can still have fun while playing and not getting bored at a certain time. If the game didn't finish at the end of the countdown, the value of all the properties, money and buildings are all added up and the player with the highest value wins.

Here is an Activity Diagram summarizing the mechanics of the game and its subtleties:

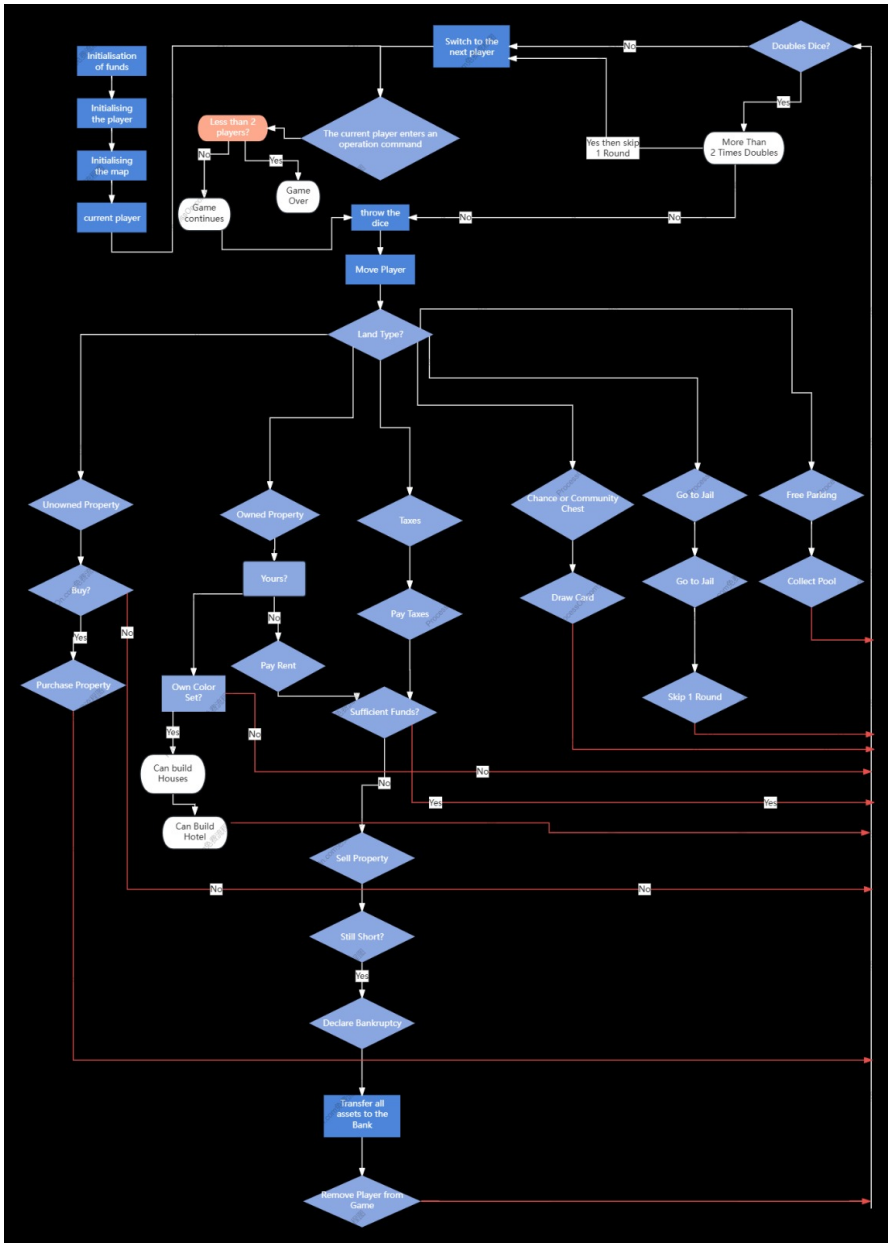

Figure 2: Activity Diagram of game mechanics

1.2 Interactions between players and server

To recreate this classic board game and adapt it to Web or mobile devices, we need to introduce our ideas to make it as functional and intuitive to every player no matter the platform they use.

Our main objective is to make the game functional for at least Desktop and mobile users, Web compatibility is an optional phase of our development process. We will explain the reasons in part **3.**

In the objective of making a fun game for every type of user who wants to share some time with friends or even strangers through open lobbies, we want the controls to be as simple as possible. For most of the actions like clicking on buttons, throwing dices or looking at particular spaces, only a simple mouse click or touch on the screen at the right spot will be needed, just simple as this.

The monopoly board being really large, it can't fit entirely in the frame of a mobile phone without making every spaces or cards unreadable, that's why we will set up a camera system automatically zooming in on the player who must play the round and the neighbor spaces on mobile platform. When a player rolls the dices, the camera zooms out to show the next spaces that are on the player's path. On their turn they can decide to zoom in by spreading the fingers, or even zoom out by pinching the screen on digital devices again to display the whole board and move around it to anticipate the possibilities or their turn so they can't have bad surprises.

To make the game as fast and smooth as possible, games will entirely be managed by the main server. Players will only be able to give instructions like "throw dice" or "buy house" to make the game work, every other aspects like consulting parameters, lowering the music volume only depend on the device of a specific player and won't go through instructions on the server.

Here is an Use case Diagram showing the interactions between a player and the server to do the main actions of the game:

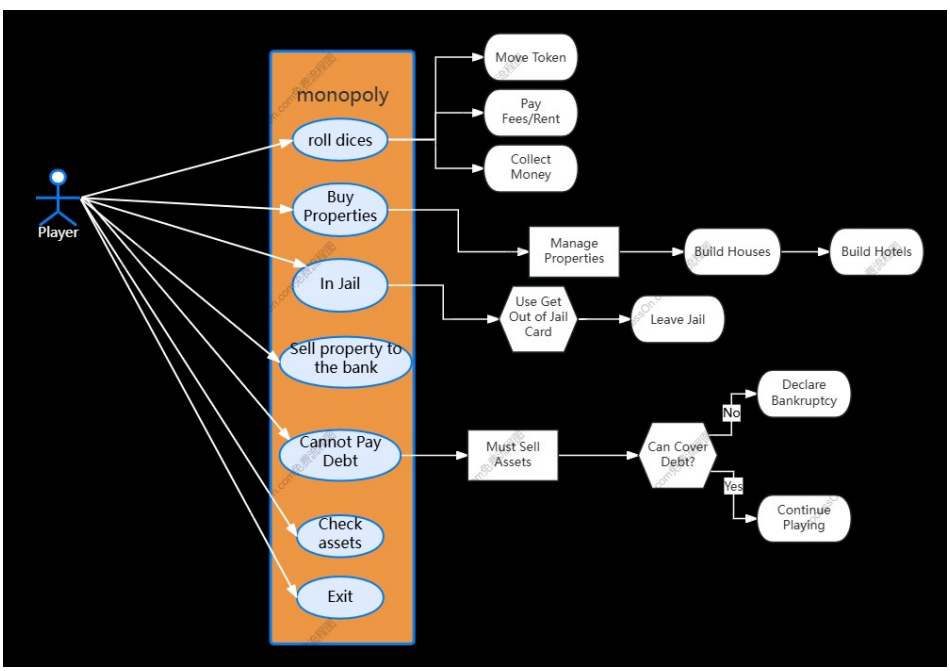

Figure 3: Use case Diagram of game mechanics

2 Key elements and choices

In this project, some key topics will be treated in certain ways according to the team's choices. That is why in this part we will aboard each one of the elements mentioned in the subject and explain our decisions applying to them in the next part:

2.1 Programming

2.1.1 Mobile development

The first main step of the project will be to implement the game mechanics and functionalities for Desktop for us to use it on our software and test it together. But for the final product, we want to adapt the interface and needed inputs for players on mobile devices to use our game easily.

As described in part **1.2**, the controls will be adapted for mobile players who will be able to tap their screen to access menus and click on buttons, and the camera will follow the player not to display the whole board on the screen and make every spaces and cards unreadable. This camera system won't be activated on Desktop devices, but on mobile players will be able to zoom in, zoom out and even freely control the camera to explore the whole board as they want.

2.1.2 Web development

In a second part, which we don't want to spend too much time on since both mobile and web aren't needed for the project, we would also like to adapt it to web devices users.

Knowing we will start developing the game with an interface made for mobile users, we will have to readapt it to players on larger non digital screens. It is why buttons and actions will be able to be done with a mouse or a touch pad. Menus will be larger and more responsive depending on the size of the window and display only primordial information with smaller frames. During a game, the board will be fully displayed the whole time. The texts and cards display will be big enough so the players can read and see everything all at once on their screen unlike mobile devices.

2.1.3 Interactions

In terms of playability, players will only have to click on certain buttons to activate particular actions to play. We want each player to be able to see the whole board to predict their moves, take a look at the properties they bought and throw dices with a simple use of a button displayed on the screen.

All the game mechanics described in part **1.1** will be managed with the server who will automatically calculate what must be paid to buy a property or to have a building, even the movements of the player's character will automatically be done. We want the controls to be as simple as it can be for everyone to understand how to play the game really easily and quickly to start their first game with their friends or other players connected to the platform.

In this type of game, players don't play together but against each other. The main goal being "the last one standing wins" implies that a player has to make others bankrupt. The game mechanics can evolve throughout the project and we can always add a "Team mode" where two players play against two others and have to collaborate to buy their properties and organize their budget together. No matter the game mechanics we want to include after, the basics stay the same.

That is why the only important interaction between other players in the game is paying rents when landing on someone else's property. In case of other game modes, we could add <u>different emojis</u> or an <u>online chat</u> for players to be able to communicate with each other not having to connect on an external website or software to do so.

2.2 System

2.2.1 Interfaces

To help users know where they can find any information or how they can play the game, interfaces will be really important in our project. Buttons will contain pictograms or texts explicitly showing the actions they are meant for. For example: parameters will be a gear, "leave the game" will be an open door and "throwing dices" will be two dices.

We will make the screen as clean as possible, paying attention to the aesthetic look of all the menus for everyone to understand clearly every part of the game. All important texts like "Create a game" or "Join a game" will be highlighted on the main screen with particular color charts following the general graphic aspect of the game (detailed in part "**Packaging**"), and all useful buttons will only be displayed when needed during the game.

2.2.2 Client / Server communications

All the information about the communications between the clients and the server are already listed in part **1.2**. But to summarize, every manipulation implying the game mechanics made by a player are only instructions sent to the server so it knows what it has to do as showed on Figure 3. Every movement, transactions or game manipulations are only made by the server itself, the player can only tell what he wants the game to do for him.

2.3 Database

A database will be really important for the kind of game we want to develop. We will need it to save users information such as id, password, displayed name, and other data if needed to identify every player when they log in again on the game.

To manage the spaces of the board and the effects of each, we will also need a database to store properties information like name, price, prices of buildings, different rents depending on the number of building, the color group it belongs to, ... But also to store the different "Chance" and "Community chests" cards because each of them has different effects on the game, the player who drew the card or other players. So we need the effects or a function pointer to be stocked somewhere and to activate the right function when needed.

Most of the interactions between the player and the server will only consists of asking for an information requested by an input, moving the character or doing the action wanted and search of the precise information in the database linked to the card drew or the space the player landed on.

2.4 Network

2.4.1 Implementation and communication protocol tests

To develop and test the communication protocol, we started making small programs to try how the Godot server system worked. The game won't be directly implemented with an external server connected to it to test the interactions and communications with the server.

Kian, a member of the server implementation team hosts a private server and can configure it to later, when the communications will work on a local scale, port the server on his own one and begin programming with an external server to implement the procedures and protocols.

Finally, everything will be migrated on the servers of the university to host the main game server and the API with the databases and adapt them to establish connections with users again.

2.4.2 Implementation of clients and server

To allow the player and the server to communicate with each other with instructions and answers showing actions and modifying the game, we thought about using WebSockets sent to the server

hosting the game, between the game and the databases there is multiple ways to do so. For now we're planning to use an API Server using Node.js, but we know it can also be done in Python language. The way we implement the servers can still evolve during the development process but the choice of the two possibilities are detailed in part **3.2**.

2.4.3 Communications safety

To secure players data especially the ones in the "Players" database, we will use HTTP protocols to authenticate players when they log into the game. The choice of this protocol is detailed in part **3.2**

2.4.4 Network's infrastructure

To summarize every aspects of the network introduced in the previous points of part **2.5** and explained in part 3.2, here is an illustration of the infrastructure of the network we would like to implement and how the servers would communicate with each other to access the database, receive players requests and do the actions intended:

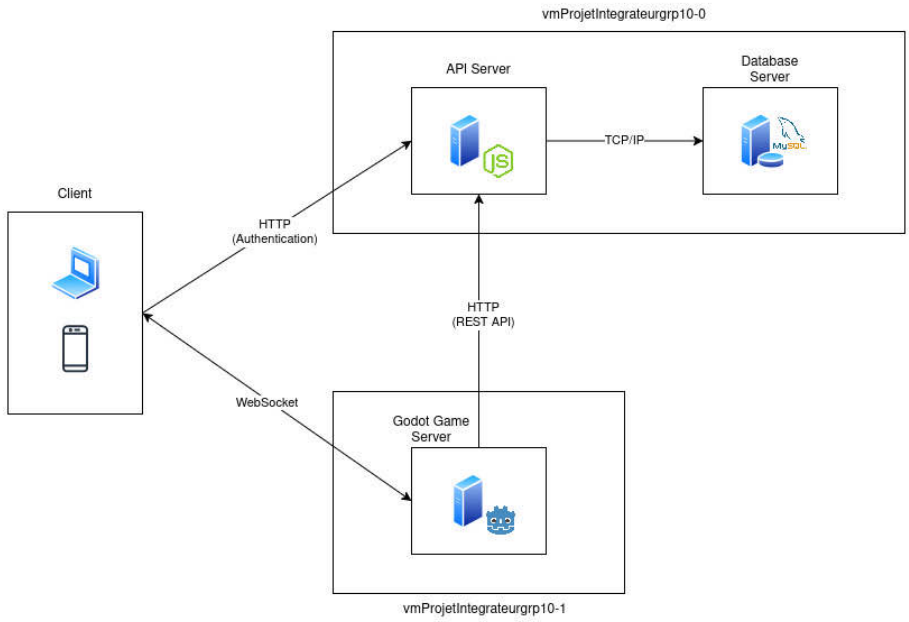

Figure 4: Illustration of the Network's infrastructure with server names

2.5 Communication / Marketing

2.5.1 Packaging

To talk about the packaging and the look of our project, we needed a theme first. It is why after a couple of meetings we decided we wanted something that linked all the members in the same idea. Being all from different countries and having different origins, we thought about a world tour themed-game.

We know many possibilities are open to us, and other games inspired by Monopoly about traveling the world already exists. Though, we don't want to look like them. The main example for this type of games is Business Tour. The design is colorful but quite simplistic and not really innovative, that is why we want to make dynamic menus and well designed backgrounds to the board and the environment not to make it look like a board in an empty space.

The buttons and interfaces will be made to suit the theme and the ambiance we want to immerge the players for an optimal experience. Following this direction, we thought about making the player's characters planes so they have the feeling to travel around the world on their own.

Figure 5: Advertisement cover for "Business Tour"

2.5.2 Communication around project

To communicate about the game, we want to create Instagram and TikTok accounts. We want the targeted audience to be around between 15 and 30 years old, so using popular medias for these generations of people seems like a good idea.

The content would be posted at a regular interval of time, and will include sneak peeks, pictures of the progress, short videos showing the first mechanics and the first animations and if we have time, interviews of team members to explain their role in the project to the followers and get to know the team.

The posts will probably exclusively be in french since we don't know if we will have time to implement multiple languages into the game, and also to be able to make advertisement in the university. After some progress about the HIM and the game, we will probably print fliers and put them around rooms or theaters at the university to show our game to the perfect targeted audience.

3 Tools and software used with justifications

3.1 Development tools

Godot and GDscript

The most important choice of this project was the language and the programming platform. Not wanting to start from scratch and have to manipulate different languages, we decided to develop our game on **Godot** which uses the **GDscript** programming language.

We chose this software because all of us heard about it at least once like being a really useful tool to develop a game like we had in mind, in full 2D or even with 3D parts, with camera movements and server interactions.
The GDscript language being often compared to Python was a good way to put our knowledge to the test while learning the few different details of this particular language and allows us to manipulate different aspects of the game with a single way to write code and not having to switch between C for the game mechanics, SQL for Database, etc.

One of our team member, Anton, already knowing the basics of the software suggested us to begin with programming for Mobile platform due to the efficiency of Godot with this type of support. Web programming can also be done but didn't sound as intuitive as mobile to design a multiplayer board-

game. Although if we still have time left at the end of the main development phase, we would like to implement a Web client allowing players to play across different platforms.

PostgreSQL

Even if we mostly learned MySQL to develop databases in previous courses, we found out about a more optimized way to program them with this language.

PostgreSQL has multiple similarities with MySQL like organizing data in form of a table and the language itself. But PostgreSQL offers better options that MySQL in some domains that are really important for our project. First of all, it allows to save data in the form of **Objects**, which means we can give them attributes and stock them together as one entity in the database. It also allows to associate functions from other languages than SQL and execute them, which is really useful for us who want particular spaces to have their own function in GDscript.

And finally it also complies with ACID properties in any configuration, contrary to MySQL which needs to be used with InnoDB or NDB Cluster software to comply to these norms.

3.2 Network tools

WebSockets

To assure the communications between the player and the server we would like to use **WebSockets** because they allow almost real-time communication which is the type of interactions we want to have for a game like Monopoly.
The protocols will be developed using Godot's language GDscript and are mostly made for external hosted servers compared to the High-Level Multiplayer API of Godot which focuses on local hosted servers and local games without external server interactions.

API Node.js

The Node.js part of API set in place was chosen to efficiently treat and modify data exploring the databases it is connected to. The main advantages of Node.js are its flexibility on mobile or web applications, its high performance treatment, doesn't need a lot of time to learn and can handle multiple requests simultaneously with non-blocking requests.

Python language

Using the Python language is an other solution to code the API communicating with the databases. In our project it also has different advantages because it is close to GDscript language and is adapted to interactions with Godot's server structure as well as multiple other languages, it can easily be transported from a platform to another and work the same on web or mobile devices and all the team members already used it for programming and saves time learning a new language in the whole programming process.

HTTP Protocols

HTTP Protocols when implemented with HTTPS adds an encryption to the data transported through the communications thanks to SSL certificates. It is also adapted to a large amount of interactions and connections on a same database and guarantee secured data transfers combined with **PostgreSQL** to communicate with databases.

3.3 Graphic tools

Draw.io

Draw.io is a useful tool to create UML diagrams to represent how a program works, how functions are supposed to work in the final product and even show the communications between clients and the server.

This tool is available online and is completly free, the interface is really intuitive and easy to learn to begin making diagrams rapidly. In comparison to other software with the same purpose, Microsoft Visio is powerful and has more features than Draw.io but is long to accommodate to and profitable.
In addition, Draw.io allows multiple team members to work together on a same project and shows the modifications of others on everyone's device, useful when teams must work together on a same task.

Figma

This online tool allows users to create wire frames to visualize the final look of their project the way they want it to look like. The main advantages of this tool is its free access just needing to create an account to navigate through the functionalities. It is also made to simulate the button's actions and create links between different wire frames to have an overall look at the project.
Multiple people can work on the same wire frame at once and collaborate together, just like Draw.io. The main problem with Figma is the possibility to create only a limited amount of projects with the free subscription. Hopefully we don't need a lot of work spaces to develop the Human Machine interfaces of our game.

Canva

Canva is an advanced image / video editing website made for companies and professional people. This tool can create dynamic and aesthetic titles as well as removing objects, add new objects to a picture or even expand an image with constantly upgraded artificial intelligence technologies.
This website will be very useful to create the logo of our game and the different posts for our social medias in addition to fliers and posters we would like to share at our university for the marketing part.

3.4 Management tools

Gantt Project

We chose this software to organize the tasks because its only purpose is to design and manage Gantt diagrams efficiently. It may not be easy to use at first sight but the project manager already learned to use this software during a previous university course.
The tasks can be organized by groups, colors and names just by clicking a few simple buttons to have a professional diagram. Although, it needs to search a bit deeper into each task to define people who are in charge of them, deadlines and other additional features. But hopefully, the most important details we need, to see our progress are easy to find and to use in this software.

Trello

This website was really useful to translate the tasks of the Gantt diagrams into cards and adding visual illustrations of deadlines, members in charge of each task and alerting someone when something is due for a certain date.
The website is easy to use and to understand and grants a free professional subscription when creating an account to explore all the possibilities of it. The cards can easily be edited and it is a great argument to keep using Trello as a task management software.

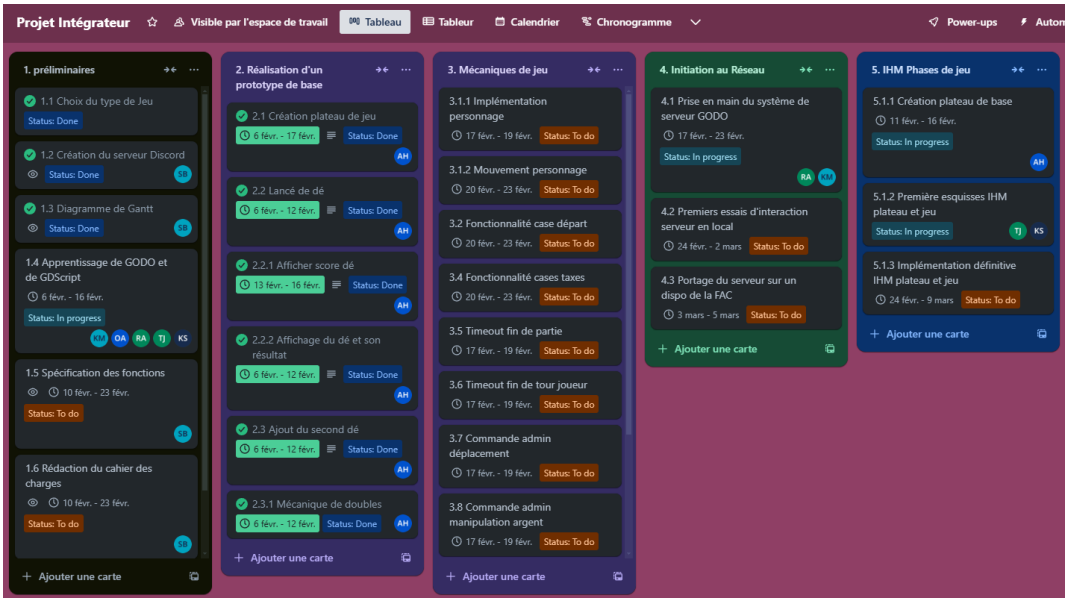

Figure 6: Screenshot of the Trello cards of our project

Google drive, docs, sheets and forms

To make the time sheets, gather the work of every team member and write the weekly recaps of each meeting, we are using most of the basic tools available on **Google Drive**.
It is a really popular website known for sharing files on an online platform with other members and to create multiple organization documents. After using it for personal planning projects, we wanted to keep track of the work and the time with it, being really useful and easy to use even for beginners.

4 Project manager's role seen by the team

To know how the group sees the role of the project manager, we thought about it and what tasks he is responsible of. In general, the team member is the coordinator of the whole project, he says what work must be done to the different teams before they split them between the different members.

He also is the spokesperson who communicates with the tutor and the teachers about the project in the name of the whole team and must follow the progress very often to stay focused on each potential mistake or misunderstanding between teams.
The project manager also has a role of time and tasks manager, he must plan the whole project by dividing it into small tasks to split the work between teams and control the quality of the product and the features really often.

But the vision of the project manager that Sacha heard the most is that the person with this role is the one who must warn the members when delays or mistakes are made. Even if he is here to manage the whole project, he must play the role the "bad guy" recalling the persons who don't do their job properly or in time.
There is a last point important to say too in our opinion: he has the entire responsibility of the project.

If in its final state the project contains big errors or doesn't satisfy the customers request, it is his fault. He told his team members to do the tasks the way he intended in some ways, he approved what the teams developed and put it in the final result so the "quality control" comes through him and is the responsible in any case of mistake reported by the customer. That is a point we consider more and more important throughout the project.

5 Work teams and tasks of the project

5.1 Different teams

To develop different parts of the project in parallel, the team was split into three work teams who will each focus on different parts and develop their tasks on their side while communicating with others to keep track of the specifications and the potential branch merges.

Here are the teams that will work on the project, and what their tasks are:

- **Team game development**: Amele Olivia, Anton, Abderaouf
  This group will be responsible of developing the game mechanics as well as the actions of the buttons, how the player can play, which input he should use to play and how the game works by itself. They will be joined at some point by the Human Machine interface team to agree on the places of each buttons and what each should do and where it should display on the screen.

- **Team server implementation**: Kian, Rarivoson
  This group will work on the interactions between the server and the different players. They will also have to implement a way to create a lobby and join it while making it visible on a connection menu and make the game work simultaneously on all connected devices at a time.

- **Team Human Machine interface**: Karim, Tingting
  This group will have to design the interface of each menu and the main game. They will draw sketches to share their vision of the final look of the game and menus then will implement the validated visuals directly into the code to make the game look like their ideas.

5.2 Tasks organization and distribution

The project is divided in **125 tasks** for now and can evolve throughout time depending on the needs of the team. To organize each of them, a Gantt diagram was realized illustrating every functionalities that must be developed and every other tasks on precise dates and periods of time.
In total, **8 key dates** were determined to give the team deadlines to implement particular functionalities for each version. Here is a description of each of them:

- February 5th, Start of project:
  Beginning of the first phase. The first meeting took place at this date and gave the group the opportunity to begin discussing about the tool and the language used for the project as well as the type of board game.

- February 17th, Start of development phase:
  Beginning of the main development period, the split teams will work on their respective tasks to develop a functional general prototype preparing the alpha version.
  Before reaching this milestone, we wanted to have a visual prototype of the board and the class of the properties made to place them directly in the scene of the board. Every team member had to learn the basics of Godot before beginning the main development phase.

- March 6, Server merge on project:
  Consists of importing the server mechanics on the project. No particular functionalities are required for the game mechanics, this milestone represents the beginning of the real implementation of the interactions between players and the server in the game and start working on an external server and not on a local instance to test it.

- March 13th, Alpha version release:
  At this date, we want to have a first functional and playable prototype of our game for one player only. Even if the server part was merged and began to be deeply developed a few days before that, we don't need the server to work neither having multiple players on the same game for this version. But even if we don't need multiplayer interactions yet, there is still some functionalities and features we want to have before releasing this first version:

  - Character visible on the board and character movements

  - Throwing dices mechanics and double score implemented
  - Character moves depending on the score of the dices
  - Timeout for player's turn and for whole game.
  - Functionalities of "Start", "Taxes", "Jail" and "Go to Jail" spaces.
  - Database of "Chance" cards and implementation of their effects.
  - Possibility to buy a property and implementation of the property group mechanics.
  - Implementation of the Human Machine interface into the game including sound effects and music.

- April 4, Server allows multiplayer:
  At this date, the server implementation would allow players to join a same game from different devices by connecting an external server and play the game. The features we want to be done with at this point are the followings:

  - Possibility to pay a rent to another player
  - Implementation of the main menu
  - Implementation of the parameters menu in the main menu and in the game.
  - Server receiving signals from a player and interpret them to do requested actions.
  - Server allow other connections on a same game.
  - When player leaves game, gives all his properties back to bank (available to buy again).
  - When all players leave a game, closes the lobby.
  - Game mechanics work for multiple players.
  - Fully implemented Human Machine interface in the game board and in the main menu.

- April 15, Beta version 0.1 release:
  For the release of this version, we want the main game mechanics to be functional and playable in multiplayer mode. The main menu and the parameters menu would also contain all the working and essential fields to customize the players experience on the menus or the main game. The main additional features added to the project at this point are multiple lobbies management by the server and the possibility to pay a rent to another player in game.

- April 29, Beta version 0.2 release:
  This version only consists of a massive resolution of problems after a long debug phase from the whole team in their respective domain. The process could be long to adjust simple functions or features so the game runs smoothly with the least amount of bugs in this release.

- May 13, 1.0 version release:
  At this date, the game should be completed in its final form. During the two weeks between this milestone and the previous one, we still have time to implement new mechanics or style features into the menus or the game itself. We wanted to focus on the main functionalities of the game first and thinking about new possibilities when we would really have time to think about it.

To summarize all the tasks of the different teams, here is the Gantt diagram in short and complete version with different colors to distinguish the tasks of each team (light blue for general tasks, magenta for game development team, green for server team, blue for HIM team and red for the admin commands to implement in the program).
The diagrams are shown in their current state and will change throughout the project depending on the potential delays:

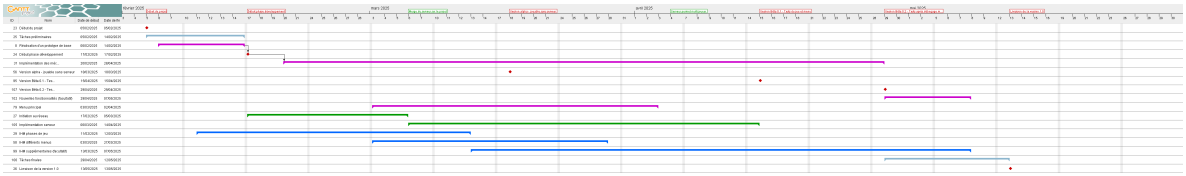

Figure 7: Short version of the current Gantt diagram of the project with main steps and milestones.

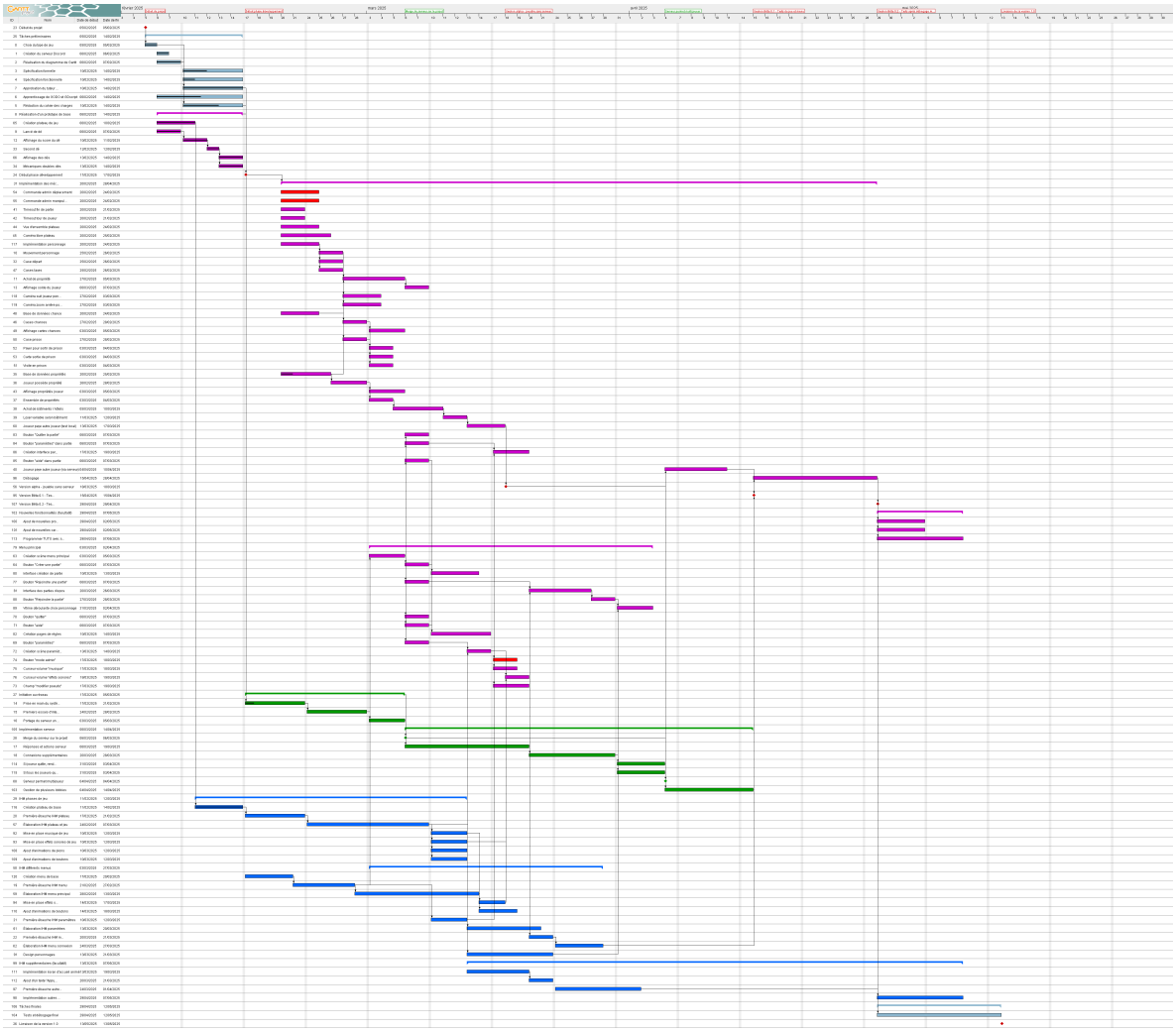

Figure 8: Full version of the current Gantt diagram of the project with every task and milestones.